

PROJECT AND TEAM INFORMATION

Project Title

Vaultify – A Secure Command-Line Password Manager in C++

Student / Team Information

<i>Team Id:</i>	<i>DSCPP-III-2026-T305</i>
Team member 1 (Team Lead) 	<i>Mishra, Ashraya</i> <i>Student ID: 2510010429</i> <i>Roll No.: 2027681</i> <i>Section : B</i>
Team member 2 	<i>Joshi, Udit</i> <i>Student ID: 2510370397</i> <i>Roll No.: 2028891</i> <i>Section : ML2</i>
Team member 3 	<i>Singh, Yuvraj</i> <i>Student ID: 2510010591</i> <i>Roll No.: 2028915</i> <i>Section: ML3</i>

PROPOSAL DESCRIPTION

Motivation

The average internet user has accounts, on websites and apps.. The human brain can only remember a few passwords well. Because of this most people. Use the same password over and over or pick simple ones that are easy to guess. This is a security problem. When a site gets hacked and passwords are leaked hackers often try those logins on other popular sites. This method, called stuffing can break into accounts that the user never meant to expose. Password managers address this problem directly by allowing a user to generate and store strong, unique passwords for every account while only needing to remember a single master password. This project aims to build a functional password manager from first principles, applying core data structures and object-oriented design concepts to solve a problem millions of people face daily.

State of the Art / Current solution

Password management today is led by tools such, as Bitwarden, LastPass and 1Password. Password management also sees browser-integrated managers built into Chrome, Firefox and Safari. These tools are convenient and widely adopted, offering features like cross-device synchronization, browser autofill, and cloud backup. However, they are largely closed-source or proprietary systems – users have limited visibility into how their credentials are actually stored, hashed, or encrypted internally, and must place complete trust in the vendor. Additionally, many advanced features are locked behind paid subscriptions, and browser-based managers tie credential storage to a single ecosystem. There is currently no lightweight, transparent, and freely inspectable alternative aimed at users or students who want to understand the underlying mechanics of secure credential storage rather than simply consuming a black-box product.

Project Goals and Milestones

The primary goal of this project is to design and implement a fully functional, secure command-line password manager in C++, applying core data structures (hash tables, stacks) and object-oriented programming principles (encapsulation, class design, file handling) taught in this course.

Core Goals:

- Implement a hash table-based vault for $O(1)$ storage and retrieval of credentials by site name
- Implement master password authentication using a hash-based verification scheme (master password is never stored in plain text)
- Implement encryption/decryption of stored passwords using a symmetric cipher keyed on the master password
- Implement persistent storage so the vault survives across sessions using file handling
- Build a clean, menu-driven CLI supporting add, retrieve, update, delete, and list operations

Stretch Goals:

- Random strong password generator
- Password strength checker for user-entered passwords
- “Recently accessed” quick-view using a stack

Milestones (aligned with course evaluation phases):

- **TA1:** Finalize project scope, system design, and technology stack; complete initial setup of core class structure (Entry, Vault, Crypto modules)
- **TA2:** Complete core functionality – hash table vault, encryption/decryption, master password authentication, file persistence, and a working CLI; begin stretch goal implementation if time permits
- **TA3:** Complete, polished, fully working project with all core features stable, stretch goals integrated where feasible, final report, and live demonstration

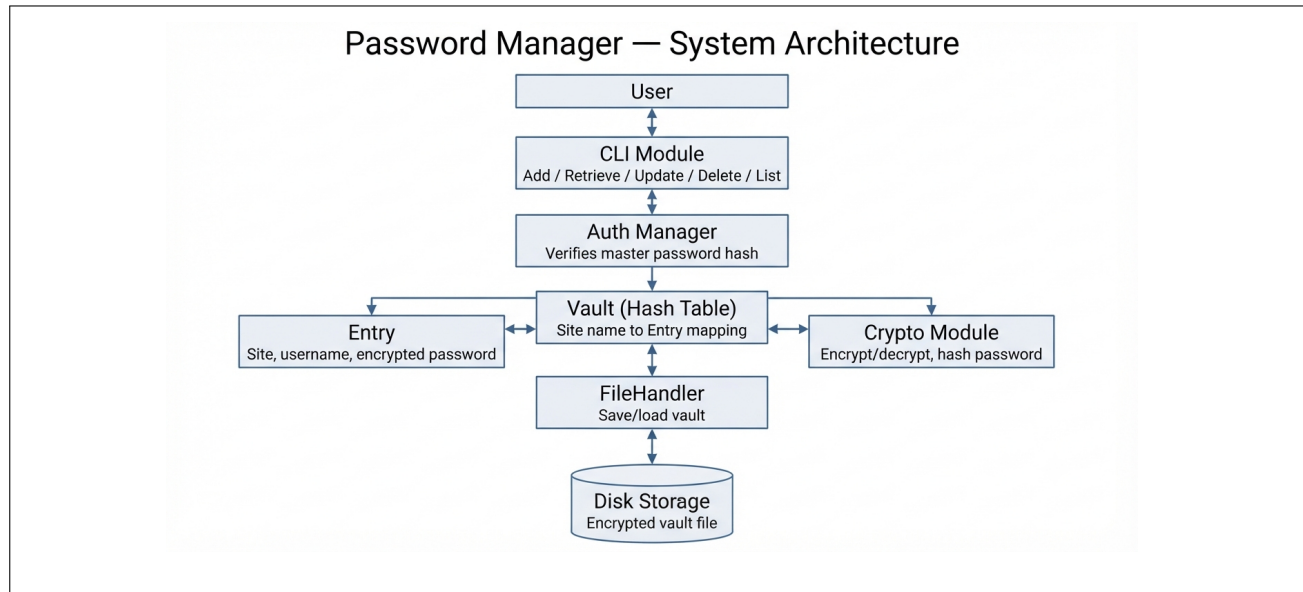
Project Approach

The project will be implemented in C++, using standard object-oriented design principles taught in this course. Development will follow a modular, bottom-up build order to ensure each component is independently testable before integration:

1. **Entry module** – defines the class representing a single credential (site name, username, encrypted password) with proper encapsulation
2. **Vault module** – implements the hash table that stores and retrieves **Entry** objects by site name, along with add/update/delete/list operations
3. **Crypto module** – implements a symmetric encryption scheme to protect stored passwords, and a hash-based verification mechanism for master password authentication
4. **FileHandler module** – handles serialization and deserialization of the vault to and from disk, ensuring persistence across sessions
5. **CLI module** – ties all components together into a menu-driven command-line interface for the end user

Development will be carried out using Visual Studio Code as the primary IDE, with GCC/g++ as the compiler. Git will be used for version control to track progress across the project phases, and the final report will be prepared using LaTeX for clean, professional formatting.

System Architecture (High Level Diagram)



Project Outcome / Deliverables

By the completion of this project, the following deliverables will be produced:

- A fully functional, cross-platform C++ command-line password manager, compiled and tested on both Windows and Linux environments
- Complete, well-documented source code implementing the Entry, Vault, Crypto, FileHandler, and CLI modules
- A compiled executable ready for direct use
- A final project report detailing the design, implementation, and technical outcomes of the project

Assumptions

This project assumes a single-user environment, where the tool is used by one individual on their own local machine, with no multi-user account support. It is assumed that the master password, once forgotten, cannot be recovered, as no password-recovery mechanism is implemented – consistent with how real password managers protect vault contents. All credential data is stored locally on disk; no cloud synchronization or network transmission is involved. Finally, this project is developed for educational purposes to demonstrate core data structures and OOP concepts, and the encryption scheme used is not intended to meet production-grade security standards.

References

1. Thareja, R. *Data Structures Using C++*, Oxford University Press – course textbook reference for hash tables, linked lists, and related data structure concepts.
2. cppreference.com – official C++ language and standard library reference, consulted for syntax and standard practices.
3. GeeksforGeeks (geeksforgeeks.org) – reference articles on hash table implementation, collision resolution, and file handling in C++.
4. OWASP Foundation (owasp.org) – Password Storage Cheat Sheet, consulted for understanding real-world best practices around password hashing and storage (used as a conceptual benchmark, not directly implemented at production strength).
5. Course lecture notes and materials – *DSA Project Based Learning with C++*.